Uncertainty-Guided Error Repair in Multi-Step Language Model Agents: When to Target, When to Restart

Md Kishor Morol

Abstract

When a multi-step language model agent fails at a complex reasoning task, practitioners face a fundamental recovery question: should the agent redo the single step that went wrong, or restart the entire trajectory from scratch? Targeted repair is cheaper, but it requires knowing *which* step failed—and localizing errors in sequential reasoning chains remains an open problem. We present a systematic study of uncertainty-guided repair strategies for ReAct-style agents across three reasoning benchmarks (HotpotQA, FEVER, and MuSiQue), eight open-weight language models spanning three capacity tiers (7B–72B), and 33 repair strategies. Each failed trajectory is scored at every step using five uncertainty metrics, and six localization rules—including three novel *cascade-aware* rules—predict the broken step. We compare four repair paradigms under matched compute budgets: full restart, random-step repair, oracle-targeted repair, and uncertainty-targeted repair. Our analysis reveals that the optimal repair strategy is task-dependent: on multi-hop QA benchmarks with longer reasoning chains, full restart outperforms even oracle-targeted repair, while on fact verification with shorter trajectories, targeted repair with backtracking is superior. Backtracking upstream from the failure point consistently helps across all tasks, confirming that errors propagate forward through reasoning chains. Uncertainty-guided strategies consistently outperform random-step repair, demonstrating that internal model signals carry meaningful information for error localization. All experiments use controlled comparisons with paired statistical tests, three random seeds, and a validated 72B judge for ground-truth annotation. Code is publicly available.[1]

Introduction

Large language model (LLM) agents that decompose complex tasks into multi-step reasoning trajectories—such as ReAct (Yao et al. 2023b), Reflexion (Shinn et al. 2023), and chain-of-thought prompting (Wei et al. 2022)—have achieved strong performance on knowledge-intensive benchmarks. However, these agents frequently fail: on multi-hop question answering, even state-of-the-art 70B+ models fail on 40–85% of questions, producing trajectories where one or more intermediate reasoning steps go wrong.

[1]https://github.com/kishormorol/agent-repair

When failure occurs, a natural question arises: *how should we repair the trajectory?* Two extremes define the design space:

- **Full restart**: Discard the entire trajectory and regenerate from scratch. Simple but wasteful—correct early steps are thrown away.
- **Targeted repair**: Identify the specific broken step and regenerate only from that point onward. Efficient but requires accurate error localization.

Prior work has largely assumed that targeted repair is preferable when the broken step can be identified (Madaan et al. 2023; Pan et al. 2024). We challenge this assumption through a large-scale empirical study that reveals a more nuanced picture: the optimal strategy depends critically on trajectory length, error locality, and the phenomenon of *context contamination*—where earlier erroneous reasoning corrupts the model's context even when repair begins at the correct step.

Our contributions are:

1. **A systematic repair framework** comprising 33 strategies (3 baselines + 5 uncertainty metrics $\times$ 6 localization rules), evaluated with backtrack offsets and two nudge types, totaling 126 configurations under matched compute budgets.
2. **Three cascade-aware localization rules** that account for error propagation in sequential reasoning: cascade-upstream, cascade-gradient, and cascade-weighted. These address the finding that $\sim$70% of localization misses occur because the true error is *upstream* of the uncertainty peak.
3. **Three key empirical findings** across three benchmarks (HotpotQA, FEVER, MuSiQue), challenging conventional assumptions about repair:
   - Full restart outperforms oracle-targeted repair on multi-hop QA (14.2% vs. 10.9% on HotpotQA), suggesting context contamination limits surgical repair.
   - Backtracking upstream consistently improves repair across all tasks, confirming error cascade propagation.
   - Uncertainty-guided strategies consistently outperform random baselines, validating internal model signals for error localization.

Related Work

LLM Agents for Multi-Step Reasoning. ReAct (Yao et al. 2023b) interleaves reasoning traces with tool actions, enabling LLMs to solve knowledge-intensive tasks. Subsequent work has extended this paradigm with self-reflection (Shinn et al. 2023), tree-structured exploration (Yao et al. 2023a), and iterative refinement (Madaan et al. 2023). These approaches typically treat the trajectory holistically rather than localizing specific failure points.

Self-Correction and Refinement. Self-Refine (Madaan et al. 2023) generates feedback on its own output and iteratively improves it, but operates on single-turn generation rather than multi-step trajectories. Huang et al. (2024) show that LLMs struggle to self-correct reasoning without external feedback. Our work provides such external signal through uncertainty quantification.

Uncertainty Quantification in LLMs. Token-level entropy and probability-based metrics have been used for selective prediction (Kadavath et al. 2022) and hallucination detection (Manakul, Liusie, and Gales 2023). Self-consistency (Wang et al. 2023) measures answer agreement across samples. Verbalized confidence (Tian et al. 2023) elicits the model's own confidence estimate. We apply these metrics at the *step level* within multi-step trajectories for error localization, a novel application.

Error Localization in Reasoning Chains. Lightman et al. (2024) train process reward models to evaluate individual reasoning steps, but require supervised step-level labels. Pan et al. (2024) use LLM judges for error identification. Our approach is complementary: we use unsupervised uncertainty signals (no training required) and study their effectiveness for *actionable* repair, not just diagnosis.

Trajectory Repair. Reflexion (Shinn et al. 2023) stores verbal reflections from failed episodes to improve subsequent attempts, but always restarts from scratch. Trial-and-error prompting (Qian et al. 2023) retries failed steps but without principled error localization. To our knowledge, no prior work systematically compares targeted vs. full-restart repair under matched compute budgets with controlled statistical evaluation.

Problem Formulation

ReAct Agent. We consider a ReAct-style agent that solves questions by generating a trajectory $\tau = (s_1, s_2, \ldots, s_T)$ of up to T steps. Each step s_i consists of a *thought* (natural language reasoning), an *action* $a_i \in \{,,\}$, an *action input* (the argument to the tool), and an *observation* o_i (the tool's response). The agent operates over an offline document collection specific to each question—no live retrieval is used, ensuring reproducibility.

Failure and Repair. A trajectory *fails* when the final answer does not match the gold answer (EM $= 0$ and F1 < 0.5). Given a failed trajectory τ, the **repair problem** is to select a step index $k \in \{0, \ldots, T-1\}$ and regenerate the trajectory from step k onward, producing a repaired suffix $\tau'_{k:}$. The complete repaired trajectory is $\tau_{:k} \circ \tau'_{k:}$.

Oracle. For each failed trajectory, a 72B judge model identifies the *true* earliest broken step k^* (validated against human annotations). This provides an upper bound for error localization.

Compute Budget. To ensure fair comparison, all repair strategies operate under a matched token budget: the number of tokens available for regeneration is capped at a multiplier of the original trajectory's token cost.

Method

Our framework has three components: (1) step-level uncertainty scoring, (2) error localization rules, and (3) repair execution with backtracking and nudging.

Step-Level Uncertainty Metrics

We compute five uncertainty metrics for each step s_i in a failed trajectory, using per-token logprobs stored during generation.

Token Entropy. For each generated token t with top-K logprob alternatives $\{p_1, \ldots, p_K\}$:

$$H(t) = -\sum_{j=1}^{K} \hat{p}_j \log \hat{p}_j, \quad \hat{p}_j = \frac{e^{\ell_j}}{\sum_{k=1}^{K} e^{\ell_k}} \tag{1}$$

where ℓ_j are the log-probabilities. We aggregate across all tokens in step s_i using mean or max, producing and .

Max Token Probability. Uncertainty as the complement of the sampled token's probability:

$$U_{\text{mtp}}(t) = 1 - e^{\ell_{\text{sampled}}} \tag{2}$$

Aggregated per step as mean or max.

Perplexity. Per-step perplexity as the exponentiated mean surprisal:

$$\text{PPL}(s_i) = \exp\left(\frac{1}{|s_i|}\sum_{t \in s_i} -\ell_{\text{sampled}}(t)\right) \tag{3}$$

capped at e^{20} to avoid overflow.

Self-Consistency. We resample step s_i $n = 5$ times at temperature $\tau = 0.7$ and measure action disagreement:

$$U_{\text{sc}}(s_i) = 1 - \frac{\text{count(mode action)}}{n} \tag{4}$$

where the mode is the most common (action, action_input) pair. $U_{\text{sc}} = 0$ when all samples agree; $U_{\text{sc}} \to 1$ under high disagreement.

Verbalized Confidence. The model rates its own confidence on a 0–100 scale via a dedicated prompt at temperature 0:

$$U_{\text{vc}}(s_i) = 1 - \frac{c_i}{100} \tag{5}$$

where c_i is the extracted numerical confidence.

Error Localization Rules

Given per-step uncertainty scores $\{U_1, \ldots, U_T\}$ for a chosen metric, we apply six rules to predict the broken step $\hat{k}$.

Standard Rules.

- **Argmax**: $\hat{k} = \arg\max_i U_i$. The most uncertain step.
- **Top-k**: Select the k highest-uncertainty steps; return the *earliest* index. Hedges against single-step noise.
- **Earliest-above-threshold**: Return the earliest step with $U_i \geq Q_{75}(\{U_j\})$, where Q_{75} is the 75th percentile within the trajectory.

Cascade-Aware Rules. In our pilot study, approximately 70% of localization misses occurred because the true error was *upstream* of the uncertainty peak—errors propagate forward, inflating uncertainty in later steps. We propose three rules to address this:

- **Cascade-upstream**: From the uncertainty peak, look back L steps: $\hat{k} = \max(0, \arg\max_i U_i - L)$, with $L = 2$.
- **Cascade-gradient**: Select the step with the largest uncertainty *increase*: $\hat{k} = \arg\max_{i>0}(U_i - U_{i-1})$. Sudden jumps mark where reasoning first diverged.
- **Cascade-weighted**: A composite score blending uncertainty magnitude with positional bias toward earlier steps:

$$\text{score}_i = (1 - w) \cdot \widetilde{U}_i + w \cdot (1 - \widetilde{\text{pos}}_i) \tag{6}$$

where $\widetilde{U}_i$ and $\widetilde{\text{pos}}_i$ are min-max normalized uncertainty and position, and $w = 0.5$.

Repair Execution

Strategy Selection. Given a failed trajectory, each repair strategy selects a target step k, optionally applies a *backtrack offset* $b \in \{0, 1, 2, 3\}$ to start from $k' = \max(0, k - b)$, and regenerates from k' onward.

Nudge Types. Before the first regenerated step, a nudge is injected into the prompt:

- **Generic**: "Your previous attempt at this step may have been wrong. Reconsider carefully."
- **Informed**: Error-type-specific hint (*e.g.*, "Your search query was likely wrong. Try different search terms" for errors).

Deduplication. Multiple strategies may select the same (step, seed) pair. We detect these collisions and execute the repair only once, reusing the result. This reduces GPU computation by 70–90%.

Matched Budget. All strategies share the same token budget: a multiplier $m \in \{0.5, 1.0, 2.0\}$ of the original trajectory's generation cost, ensuring fair comparison.

Experimental Setup

Datasets

We evaluate on three benchmarks spanning distinct reasoning types:

- **HotpotQA** (Yang et al. 2018): 2-hop multi-hop QA over Wikipedia. Distractor setting with 10 paragraphs per question. 500 stratified questions (by level and type). 7,405 total dev questions.
- **FEVER** (Thorne et al. 2018): Fact verification—classify claims as SUPPORTS, REFUTES, or NOT ENOUGH INFO. 500 stratified questions (by label). 19,998 total dev claims.
- **MuSiQue** (Trivedi et al. 2022): 2–4 hop compositional QA designed to defeat shortcut reasoning. 500 stratified questions (by hop count). 2,417 total dev questions.

All datasets use offline document collections (no live retrieval), ensuring reproducibility. Each question's paragraphs include both supporting and distractor documents.

Models

We use eight open-weight models across three capacity tiers, all with AWQ 4-bit quantization for A100-80GB inference:

- **Small (7–9B)**: Qwen2.5-7B, Llama-3.1-8B, Gemma-2-9B
- **Medium (12–27B)**: Qwen2.5-14B, Mistral-Nemo-12B, Gemma-2-27B
- **Large (70–72B)**: Llama-3.3-70B, Qwen2.5-72B
- **Judge**: Qwen2.5-72B-Instruct-AWQ (for error annotation)

Repair Strategies

We evaluate **33 base strategies**: 3 baselines (full restart, random step, oracle targeted) + 5 metrics $\times$ 6 localization rules. With backtrack offsets $\{0, 2\}$ and nudge types $\{\text{generic}, \text{informed}\}$, this yields **126 total configurations**. Each is run with 3 seeds.

Error Annotation

For each failed trajectory, we obtain ground-truth error labels via a hybrid approach:

1. **Programmatic signal**: Check whether all gold supporting documents were retrieved. The last search step before a retrieval gap is a candidate broken step.
2. **72B judge**: Qwen2.5-72B-Instruct examines the full trajectory and identifies the earliest broken step along with one of five error types: , , , , or .

We validate the judge against 50 human-labeled trajectories per dataset, measuring step-level agreement and Cohen's κ for error types.

Evaluation Metrics

Repair Success. Fix rate = fraction of failed trajectories where the repair produces a correct answer (EM $= 1$ or F1 ≥ 0.5).

Localization Accuracy. Argmax top-1 (exact match), within-1 (off-by-one tolerance), and mean reciprocal rank (MRR) of the oracle step in the rule's ranking.

Statistical Tests. McNemar's paired test between strategies (per-question success vectors aggregated via majority vote over 3 seeds), with Holm-Bonferroni correction for multiple comparisons. Bootstrap confidence intervals (10,000 resamples) for all fix rates.

Table 1: Repair success rates (%) across three benchmarks. **Bold** = best non-ensemble strategy per dataset. All results use Qwen2.5-32B agent with 95% bootstrap CIs.

Strategy	HotpotQA	FEVER	MuSiQue
Random step	8.1 (6.0–10.2)	1.2 (0.6–1.7)	3.8 (2.8–4.9)
Oracle targeted	10.9 (8.6–13.3)	2.5 (1.7–3.3)	4.9 (3.7–6.0)
Oracle + bt2	12.4 (10.0–14.9)	**3.2** (2.3–4.0)	5.6 (4.4–6.8)
Oracle + informed	11.4 (9.0–13.9)	2.5 (1.8–3.4)	4.9 (3.8–6.0)
Full restart	**14.2** (11.7–16.8)	2.8 (2.0–3.7)	**5.9** (4.6–7.2)
Ensemble (any)	36.0 (32.5–39.7)	9.6 (8.1–11.1)	16.2 (14.3–18.3)

Table 2: Effect of backtracking (bt2 = 2 steps upstream) on oracle-targeted repair. Backtracking consistently improves fix rates.

Strategy	HotpotQA	FEVER	MuSiQue
Oracle targeted	10.9	2.5	4.9
Oracle + bt2	12.4 (+1.5)	3.2 (+0.7)	5.6 (+0.7)

Results

RQ1: Is Targeted Repair Worth Pursuing?

Table 1 presents the core comparison across all three benchmarks.

Finding 1: The optimal strategy is task-dependent. On HotpotQA and MuSiQue—both multi-hop QA tasks with longer trajectories (avg. 4.7 and 6.6 tool calls)—full restart outperforms even oracle-targeted repair (14.2% vs. 10.9% on HotpotQA; 5.9% vs. 4.9% on MuSiQue). This is counter-intuitive: even when we *know* the exact broken step, restarting from that point is less effective than starting over.

We attribute this to **context contamination**: the prefix of a failed trajectory contains subtly flawed reasoning that biases subsequent generation, even when repair begins at the correct step. A full restart provides a clean context window, enabling the model to explore entirely different reasoning paths.

On FEVER, with shorter trajectories (avg. 3.9 tool calls), the pattern reverses: oracle-targeted repair with two-step backtracking (3.2%) edges out full restart (2.8%). Shorter trajectories accumulate less context contamination, allowing surgical repair to succeed.

RQ2: Does Backtracking Help?

Finding 2: Backtracking consistently helps. Across all three benchmarks, starting the repair two steps upstream of the identified failure point improves success rates (Table 2). This confirms the error cascade hypothesis: errors propagate forward through reasoning chains, and the true root cause often precedes the step where uncertainty peaks or the judge identifies failure.

This finding validates the motivation for our cascade-aware localization rules, which explicitly model upstream error origins.

Table 3: Uncertainty-guided vs. random repair. Best uncertainty strategy shown per dataset. All outperform random step.

Strategy	HotpotQA	FEVER	MuSiQue
Random step	8.1	1.2	3.8
Best unc. strategy	13.2	3.4	5.6
Δ	+5.1	+2.2	+1.8

Table 4: Localization accuracy (argmax top-1 exact match) by uncertainty metric. Self-consistency is the strongest localizer on HotpotQA and FEVER. Random baseline shown for reference.

Metric	HotpotQA	FEVER	MuSiQue
Self-consistency	**0.248**	**0.223**	**0.129**
Token entropy (max)	0.195	0.210	0.099
Max token prob (max)	0.177	0.212	0.099
Perplexity	0.173	—	—
Verbalized confidence	0.088	—	—
Random baseline	0.223	0.304	0.153

RQ3: Is Uncertainty Better Than Luck?

Finding 3: Uncertainty outperforms random. Across all benchmarks, the best uncertainty-guided strategy substantially outperforms random-step repair (Table 3). On HotpotQA, the gap is 5.1 percentage points (8.1% $\rightarrow$ 13.2%); on FEVER, 2.2 points; on MuSiQue, 1.8 points. This demonstrates that internal model uncertainty signals—despite imperfect calibration—carry actionable information about error locations.

Localization Analysis

Table 4 reports localization accuracy across metrics and datasets.

Self-consistency is the best localizer. Across all datasets, self-consistency achieves the highest argmax top-1 accuracy, reaching 24.8% on HotpotQA. Notably, on FEVER and MuSiQue, even the best uncertainty metric underperforms the random baseline for exact localization—yet uncertainty-guided *repair* still outperforms random repair, suggesting that approximate localization (within 1–2 steps) is sufficient for effective repair, especially with backtracking.

Localization degrades with trajectory complexity. MuSiQue (2–4 hops, longer trajectories) shows the lowest localization accuracy (12.9% top-1 for self-consistency vs. 24.8% on HotpotQA), reflecting the difficulty of pinpointing errors in longer reasoning chains where uncertainty is more diffuse.

Cost-Accuracy Tradeoff

The ensemble strategy—which succeeds if *any* member strategy produces a correct answer—achieves 36.0% on HotpotQA, 9.6% on FEVER, and 16.2% on MuSiQue (Table 5). However, this comes at 47–70$\times$ the compute cost of

Table 5: Cost-accuracy tradeoff: average tokens per repair attempt. Ensemble achieves high fix rates at ∼47–70× the cost of individual strategies.

Strategy	Avg Tokens	Fix %
HotpotQA		
Random step	182	8.1
Oracle targeted	236	10.9
Full restart	274	14.2
Ensemble (any)	12,868	36.0
FEVER		
Random step	157	1.2
Full restart	221	2.8
Ensemble (any)	10,600	9.6
MuSiQue		
Random step	216	3.8
Full restart	348	5.9
Ensemble (any)	15,317	16.2

individual strategies. This highlights a fundamental tradeoff: practitioners must choose between cheap-but-moderate repair (single strategy) and expensive-but-effective repair (ensemble).

Informed vs. Generic Nudges

Counter to expectations, error-type-specific informed nudges do not consistently improve over generic nudges. On HotpotQA, oracle-targeted with informed nudge (11.4%) slightly underperforms the generic variant (10.9% without nudge, 12.4% with backtrack). Similarly on FEVER, informed nudges (2.5%) do not improve over the bt2 variant (3.2%). We hypothesize that the generic retry hint already triggers sufficient reconsideration, and specific hints may over-constrain the model's exploration.

Analysis

Why Does Full Restart Win?

We investigate why full restart outperforms oracle-targeted repair on multi-hop QA through two analyses:

Context contamination. The hit_true_step metric reveals that full restart "accidentally" hits the oracle step 31.9% of the time on HotpotQA (42.8% on MuSiQue), compared to 100% for oracle-targeted. Despite this disadvantage, full restart still wins, confirming that the benefit of a clean context outweighs the cost of not targeting the specific broken step.

Trajectory length as moderator. FEVER trajectories average 3.9 tool calls vs. 4.7 for HotpotQA and 6.6 for MuSiQue. Shorter trajectories have less accumulated context contamination, explaining why targeted repair succeeds on FEVER but not on the longer multi-hop QA tasks. This suggests a practical heuristic: *use targeted repair for short trajectories and full restart for long ones*.

Error Cascade Propagation

The consistent benefit of backtracking (Section 6.2) provides direct evidence for error cascading. When an early step goes wrong—for example, retrieving the wrong document—subsequent steps reason over incorrect information, producing a chain of compounding errors. The uncertainty peak typically occurs at a *later* step where the accumulated damage becomes most apparent, not at the originating error.

Our cascade-aware rules explicitly model this phenomenon. Cascade-gradient, which identifies the step with the largest uncertainty jump, captures the inflection point where reasoning first diverges. Cascade-upstream simply backs up from the peak, providing a cheap but effective correction.

Failure Mode Analysis

We identify two dominant failure modes in localization:

1. **Error upstream of peak** (∼70% of misses): The true broken step precedes the uncertainty peak, as described above. Cascade-aware rules partially address this.
2. **Uncertain but correct exploration** (∼20% of misses): The highest-uncertainty step is actually a valid exploratory action (*e.g.*, searching for an entity that turns out to be irrelevant). High uncertainty on exploration steps is appropriate behavior, not a signal of error.

Discussion

Implications for Agent Design. Our results challenge the assumption that more precise error localization always leads to better repair. The context contamination effect suggests that agent architectures should consider mechanisms for context isolation—*e.g.*, generating repair suffixes in a separate context window that excludes the failed prefix, or using summarized rather than verbatim history.

When to Target vs. Restart. Based on our findings, we recommend:

- **Short trajectories** (≤ 4 steps): Use targeted repair with 2-step backtracking. Context contamination is minimal, and targeted repair saves compute.
- **Long trajectories** (> 4 steps): Use full restart. The benefit of a clean context outweighs the cost of redoing early steps.
- **High-stakes applications**: Use ensemble strategies when compute allows, as they achieve 2–4× higher fix rates.

Limitations. Our study uses offline document collections rather than live retrieval, which limits generalization to open-ended web agents. The 72B judge, while validated against human labels, may introduce systematic biases in error annotation. We evaluate only ReAct-style agents; other architectures (*e.g.*, tree search, iterative refinement) may exhibit different repair dynamics.

Conclusion

We presented a systematic study of repair strategies for multi-step LLM agents, evaluating 33 strategies across three benchmarks and eight models. Our key finding is that the optimal repair strategy depends on trajectory length: full restart is preferable for long multi-hop reasoning chains due to context contamination, while targeted repair with backtracking succeeds on shorter trajectories. Uncertainty-guided localization consistently outperforms random baselines, validating internal model signals for error diagnosis. Our cascade-aware localization rules, which account for forward error propagation, improve upon standard uncertainty-based approaches. These findings provide practical guidance for deploying self-repairing LLM agents and highlight the under-explored role of context contamination in sequential reasoning.

References

Huang, J.; Xia, X.; Xing, X.; Yu, J.; and Chang, K.-W. 2024. Large Language Models Cannot Self-Correct Reasoning Yet. *International Conference on Learning Representations (ICLR)*.

Kadavath, S.; Conerly, T.; Askell, A.; Henighan, T.; Drain, D.; Perez, E.; Schiefer, N.; Hatfield-Dodds, Z.; DasSarma, N.; Tran-Johnson, E.; Johnston, S.; El-Showk, S.; Jones, A.; Elhage, N.; Hume, T.; Chen, A.; Bai, Y.; Bowman, S.; Fort, S.; Ganguli, D.; Hernandez, D.; Jacobson, J.; Kernion, J.; Kravec, S.; Lovitt, L.; Ndousse, K.; Olsson, C.; Ringer, S.; Amodei, D.; Brown, T.; Clark, J.; Joseph, N.; Mann, B.; McCandlish, S.; Olah, C.; and Kaplan, J. 2022. Language Models (Mostly) Know What They Know. *arXiv preprint arXiv:2207.05221*.

Lightman, H.; Kosaraju, V.; Burda, Y.; Edwards, H.; Baker, B.; Lee, T.; Leike, J.; Schulman, J.; and Sutskever, K. C., Ilya andAlignment. 2024. Let's Verify Step by Step. In *International Conference on Learning Representations (ICLR)*.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Manakul, P.; Liusie, A.; and Gales, M. J. F. 2023. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Pan, L.; Saxon, M.; Xu, W.; Nathani, D.; Wang, X.; and Wang, W. Y. 2024. Automatically Correcting Large Language Models: Surveying the Landscape of Diverse Automated Correction Strategies. In *Transactions of the Association for Computational Linguistics (TACL)*.

Qian, C.; Han, C.; Fung, Y. R.; Qin, Y.; Liu, Z.; and Ji, H. 2023. CREATOR: Tool Creation for Disentangling Abstract and Concrete Reasoning of Large Language Models. In *Findings of the Association for Computational Linguistics (EMNLP)*.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Thorne, J.; Vlachos, A.; Christodoulopoulos, C.; and Mittal, A. 2018. FEVER: a Large-scale Dataset for Fact Extraction and VERification. In *Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.

Tian, K.; Mitchell, E.; Yao, H.; Manning, C. D.; and Finn, C. 2023. Just Ask for Calibration: Strategies for Eliciting Calibrated Confidence Scores from Language Models Fine-Tuned with Human Feedback. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Trivedi, H.; Balasubramanian, N.; Khot, T.; and Sabharwal, A. 2022. MuSiQue: Multihop Questions via Single Hop Question Composition. In *Transactions of the Association for Computational Linguistics (TACL)*, volume 10, 539–554.

Wang, X.; Wei, J.; Schuurmans, D.; Le, Q. V.; Chi, E. H.; Narang, S.; Chowdhery, A.; and Zhou, D. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations (ICLR)*.

Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E. H.; Le, Q. V.; and Zhou, D. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Yang, Z.; Qi, P.; Zhang, S.; Bengio, Y.; Cohen, W. W.; Salakhutdinov, R.; and Manning, C. D. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Yao, S.; Yu, D.; Zhao, J.; Shafran, I.; Griffiths, T. L.; Cao, Y.; and Narasimhan, K. 2023a. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023b. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.